

SOPHIX V3 DD Index And Implementation Guardrails

Version: v1.1

Status: Current navigation and coherence control

Date: 2026-06-01

Purpose: give dev leads one short entry point into the DD set and record the canonical decisions that prevent duplicate modules, overdesigned workers, and unclear frontend/backend ownership.

Use this file with DD_CROSS-00-System_Traceability_and_Implementation_Map-v1.0.md. The cross-map is the detailed feature-to-module traceability document; this index is the quick control page.

Binding implementation baseline:

- 99_analysis/Sophix_V3_Performance_Complexity_Assessment_20260601.md
- 99_analysis/Sophix_V3_MVP_Runtime_Simplification_Baseline_20260601.md

When a DD appears to imply one deployment per DD, one worker pod per topic, or full first-release scope for an advanced process, the simplification baseline is authoritative unless architecture governance approves an exception.

1. Bundle Map

Bundle	Main content
01_foundations	Auth/IDP, DB, Kafka, cache, Camunda, Drools, file storage, shared runtime standards.
02_framework	Subscription lifecycle framework and shared lifecycle management.
03_subscription_workflows / 04_subscription_workflows	Subscription activation, pause, resume, terminate, upgrade, downgrade, restrict, suspend, relocate, migrate.
05_billing	Charging, invoicing, payments, wallet, dunning, rating/mediation/payment gateway where applicable.
06_fulfillment	Order fulfillment, activation/restriction/termination fulfillment, HomePass lifecycle, provisioning integration, anchor/equipment/technology migration fulfillment satellites.
07_work_order	Work order lifecycle, dispatch, assignment, execution, field finalization, subscription-to-WO contract.
08_osr_equipment	Stock, equipment instances, RMA/swap, procurement, physical inventory audit.
09_catalogs	Customer/account, HomePass, technical regions, package/service/product/tax/wallet/discount/equipment catalogs.
10_other	Ticketing, service assurance, franchise/sales, RBAC, approvals, contractor/staff, notifications, reporting, CVM, field audits, customer timeline.
99_frontend	Shared frontend foundation, backoffice, outdoor sales, contractor mobile, customer self-care, reporting portal, USSD channel.
99_analysis	Review reports, gap reconciliation, process coverage register, hardening notes.

2. Canonical Ownership Decisions

Topic	Canonical decision
Contractor/staff registry	DD_EM-02-Contractor_and_Staff_Registry-v1.0.md is the implementation target. DD_EM-02-Contractor_Team_Agent_Registry-v1.0.md is superseded and kept only for traceability.
Ticketing/service assurance	TCK-01 is the ticket/case engine. ASR-01/02/03/04 are flow/type satellites on top of TCK, not a second ticketing service.
Subscription migration	SUB-WF-MIGRATION-01 owns the business operation and subscription state transition. FUL-10 owns cross-technology technical fulfillment.
Work orders	WO-01 owns work order lifecycle and execution. Other modules may request WOs but must not write WO lifecycle tables.
Equipment	OSR owns equipment instance and stock truth. WO stores field evidence; FUL may orchestrate service rebinding; neither becomes the stock ledger.

Topic	Canonical decision
Field audits	FA-01/02/03 are audit process types. Implement as one audit capability with <code>audit_type</code> unless scale later requires split deployments.
Reporting	REP-01 owns reporting mart/API. FE-APP-05 consumes REP-01 and may link to operational read APIs for drill-down only.
USSD	FE-CH-USSD-01 is a channel adapter over existing module/self-care APIs. It owns menu/session/audit state only.
RBAC	FOUNDATION_AUTH handles identity/token integration. EM-CFG-03 owns business roles, permissions, scopes, and frontend visibility mapping.
Approvals	EM-CFG-04 owns reusable approval definitions. Owning modules keep their own approved business state and audit.

3. Runtime Guardrails

3.1 Workers

Worker names in DDs are logical task contracts, not mandatory pods. A single worker application may handle many low-volume topics if metrics, retries, logs, and dead-letter handling remain visible per topic.

Start with a small set of worker apps: subscription operations, billing operations, field operations, equipment operations, integration, and reporting/CVM. Split only for throughput, dependency isolation, security, or team ownership.

3.2 Camunda

Use Camunda for long-running workflows with timers, user tasks, approvals, compensation, retries, and visible operation state.

Do not use Camunda for simple CRUD, request validation, read-only aggregation, or deterministic scans that can be scheduled commands/jobs.

3.3 Drools

Use Drools for operator-configurable policy: eligibility, routing, approval requirement, discounts, dunning thresholds, field-audit discrepancy treatment, and similar business decisions.

Keep fixed validation in service code: required fields, enum checks, FK/existence checks, idempotency, unique constraints, and simple permission guards.

4. MVP Runtime Baseline

For the first implementation wave, deploy capability groups rather than individual DDs:

Group	Capability examples
Customer/catalog API	ILM, PLM, SIP, RLM, technical-region/config APIs.
Subscription API/workflow	SUB-LM, SUB-WF, subscription lifecycle workers.
Billing API/jobs	charging, payment, invoice, wallet, dunning, tax, rating batch interfaces.
Fulfillment API/workflow	FUL-02/03/04/05/07/08/09/10, provisioning orchestration.
Field operations API	WO, EM-02, field audit.
Equipment/stock API	OSR stock, equipment instance, RMA, procurement, audit.
CRM/commercial API	TCK, ASR types, SALES, EM-01, campaigns, discounts, CVM.
Integration workers	payment gateway, provisioning, tax gateway, mediation/rating, external callbacks.
Reporting API	REP-01 dashboards, exports, reconciliation.

Do not split a group until throughput, security, scaling, release independence, or team ownership requires it.

5. Frontend/API Rule

Every frontend screen must identify:

- owning module for every command;

- supporting read APIs for every panel;
- role/permission from EM-CFG-03 and FOUNDATION_AUTH;
- idempotency behavior for state-changing actions;
- file upload/download path through file storage;
- operation-status or event/polling behavior for long-running actions.

If a screen cannot identify an owning module API, the design is not ready for implementation.

6. Dev Guide Endpoint Note

Some generated implementation guides include an `Endpoint scan` reference row in the runtime metadata table. Treat that row only as an extracted endpoint hint, not as the primary business command. The actual implementation order is defined by the DD's API section, request/response examples, and execution-flow guide.

7. Remaining Implementation Deliverables

The DD set is now broad enough for squad planning, but implementation still needs these concrete deliverables:

- per-module OpenAPI files generated from the DD contracts;
- automated contract tests for each published API;
- cross-module E2E tests from DD_QA-01;
- HLD and process-map PDFs refreshed to match the final DD set;
- regenerated PDF/zip review pack after this coherence pass.

8. Dev Reading Order

For a new squad, read in this order:

1. DD_INDEX.md and DD_CROSS-00-System_Traceability_and_Implementation_Map-v1.0.md.
2. Foundation DDs for auth, DB, Kafka, cache, Camunda, Drools, file storage, and API standards.
3. The owning module DD and its dev implementation guide.
4. Any upstream/downstream DD listed in the module's cross-DD coordination section.
5. The frontend DD that will consume the module API.
6. DD_QA-01 scenarios that cover the feature.